PySpark入门

(开始时间：2021/12/13)
(结束时间：2021/12/21)
(0：10)

(by 孟智超)

1 CATLOG

1 CATLOG
2 一、pyspark使用语法
3 二、RDD的基本操作
3.1 RDD介绍
3.2 RDD基本操作以及示例
3.2.1 count()
3.2.2 collect()
3.2.3 foreach(func)
3.2.4 filter(f)
3.2.5 map(f, preservesPartitioning = False)
3.2.6 reduce(f)
3.2.7 join(other, numPartitions = None)
3.2.8 distinct()
3.2.9 randomSplit()
3.2.10 groupBy()
3.3 RDD集合运算
3.3.1 union()
3.3.2 intersection()
3.3.3 subtract()
3.3.4 cartesian()
3.4 RDD基本动作运算
3.4.1 读取元素
3.4.2 统计功能
3.4.3 单个RDD Key-Value基本“转换”运算
3.4.4 多个RDD Key-Value“转换”运算
3.4.5 持久化操作
4 三、DataFrame的基本操作
4.1 查的操作
4.1.1 行元素查询操作
4.1.2 列元素查询
4.2 增改的操作
4.2.1 新建、新增数据
4.3 合并、集合的操作
4.3.1 union()
4.3.2 join()
4.3.3 subtract()、intersect()、union()
4.4 explode()与split()
4.5 统计的操作
4.5.1 df.stat.freqItems()
4.5.2 crosstab()
4.5.3 groupby()
4.5.4 Pivot()
4.6 删除的操作

4.6.1 drop()
4.6.2 sparkDataFrame.na.drop()
4.6.3 sparkDataFrame.dropna()
4.6.4 fillna()、.na.fill()
4.7 格式转换的操作
4.7.1 Pandas和Spark的DataFrame的相互转换
4.7.2 RDD与Spark dataframe转换
4.8 sql操作
4.9 读写csv的操作
4.10 pyspark.sql.functions
4.10.1 udf针对一列操作
4.10.2 udf()针对多列操作
4.11 字符串方法
4.11.1 concat、concat_ws
4.11.2 格式化字符串
4.11.3 instr
4.11.4 substring
4.11.5 regexp_repalce
4.12 与时间相关的方法
4.12.1 格式更改、日期获取
4.12.2 日期、月份差
4.13 窗口函数的使用
4.14 其他函数

2 一、pyspark使用语法

- 在Spark2.0之前， SparkContext 是所有 Spark 功能的结构， 驱动器（driver） 通过 `SparkContext` 连接到集群 （通过resource manager）， 因为在2.0之前， `RDD` 就是Spark的基础。如果需要建立 `SparkContext` ，则需要SparkConf，通过Conf来配置SparkContext的内容。

```
from pyspark import SparkConf,SparkContext
conf = SparkConf().setAppName('app').setMaster('local')
sc = SparkContext(conf=conf)
```

  其中：

  1. `setAppName()` , 是你的程序在集群上的名字
  2. `setMaster()` , 你的Spark运行的模式 'local'表示本地模式

     而在Spark2.0之后，Spark Session也是Spark 的一个入口， 为了引入dataframe和dataset的API， 同时保留了原来SparkContext的functionality， 如果想要使用HIVE，SQL，Streaming的API， 就需要Spark Session作为入口。

```
spark = SparkSession.builder.appName('testSQL')\
                      .config('spark.some.config.option','some-value')\
                      .getOrCreate()
```

- 写一个小小的例子，创建一个 `SparkContext` 对象并统计一个文件中的行数。

```
# 创建一个SparkContext对象并统计一个文件中的行数。

from pyspark import SparkContext
logFile = r"C:\Users\Administrator\Desktop\sklearn\sklearn.md"
sc = SparkContext("local", "first app")
logData = sc.textFile(logFile).count()
print(logData)
```

3 二、RDD的基本操作

3.1 RDD介绍

- Spark的核心是RDD（Resilient Distributed Dataset）即弹性分布式数据集，属于一种分布式的内存系统的数据集应用。Spark主要优势就是来自RDD本身的特性，RDD能与其他系统兼容，可以导入外部存储系统的数据集，例如，HDFS、HBase或者其他Hadoop数据源。它们是在多个节点上运行和操作以在集群上进行并行处理的元素。RDD是不可变元素，这意味着一旦创建了RDD，就无法对其进行更改。RDD也具有容错能力，因此在发生任何故障时，它们会自动恢复。你可以对这些RDD应用多个操作来完成某项任务。
- 要对这些RDD进行操作，有两种方法
    1. `Transformation`： 转换 - 这些操作应用于RDD以创建新的RDD。Filter，groupBy和map是转换的示例。
    2. `Action`： 操作 - 这些事应用于RDD的操作，它指示Spark执行计算并将结果发动回驱动程序。
- 使用 `sparkcontext` 读取文件：
    1. 读入文件内容，返回的东西是rdd

    ```
    # 读入文件内容，返回的东西是rdd
    data_rdd = sc.textFiles(r"C:\Users\Administrator\Desktop\sklearn\sklearn.md")
    ```

    2. 不仅读入文件内容，还会读入文件的路径path

    ```
    # 不仅读入文件内容，还会读入文件的路径path
    path_data_rdd = sc.wholeTextFiles((r"C:\Users\Administrator\Desktop\sklearn\sklearn.md"))
    ```

3.2 RDD基本操作以及示例

- `error`

  解决： `sc.stop()`

  > ValueError: Cannot run multiple SparkContexts at once; existing SparkContext(app=count app, master=local) created by **init** at :3

3.2.1 count()

- 返回 `RDD` 中的元素个数

```python
from pyspark import SparkContext

sc = SparkContext('local','count app')
words = sc.parallelize(
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"
     ])
counts = words.count()
print(f'number if elements in RDD ->{counts}')


"""输出"""
number if elements in RDD ->8
```

3.2.2 collect()

- 返回 `Rdd` 中的所有元素，即转换为python数据类型

```python
from pyspark import SparkContext

sc = SparkContext('local','collect app')
words = sc.parallelize(
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"
     ])

coll = words.collect()
print(f'elements in RDD: \n {coll}')
print(f'type of coll:{type(coll)}')


"""输出"""
elements in RDD:
 ['scala', 'java', 'hadoop', 'spark', 'akka', 'spark vs hadoop', 'pyspark', 'pyspark and spark']
type of coll:<class 'list'>
```

3.2.3 foreach(func)

- 仅返回满足 `foreach` 内函数条件的元素。在下面的示例中，我们在 `foreach` 中调用 `print` 函数，该函数打印 `RDD` 中的所有元素。

```
from pyspark import SparkContext

sc = SparkContext('local', 'ForEach app')
words = sc.parallelize(
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"
     ])

def f(x): print(x)

fore = words.foreach(f)

"""输出"""
scala
java
hadoop
spark
akka
spark vs hadoop
pyspark
pyspark and spark
```

3.2.4 filter(f)

- 返回一个包含元素的新 `RDD`，它满足过滤器内部的功能。在下面的示例中，我们过滤掉包含'' `spark` '的字符串。

```
sc.stop()
from pyspark import SparkContext
sc = SparkContext("local", "Filter app")
words = sc.parallelize(
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"]
)
words_filter = words.filter(lambda x: 'spark' in x)
filtered = words_filter.collect()
```

```
print(f"Fitered RDD :{filtered}")



"""输出"""
Fitered RDD :['spark', 'spark vs hadoop', 'pyspark', 'pyspark and spark']
```

3.2.5 map(f, preservesPartitioning = False)

- 通过将该函数应用于 `RDD` 中的每个元素来返回新的 `RDD` 。在下面的示例中，我们形成一个键值对，并将每个字符串映射为值1

```
sc.stop()
from pyspark import SparkContext
sc = SparkContext("local", "Map app")
words = sc.parallelize(
    ["scala",
     "java",
     "hadoop",
     "spark",
     "akka",
     "spark vs hadoop",
     "pyspark",
     "pyspark and spark"]
)
words_filter = words.map(lambda x: (x,1))
mapping = words_filter.collect()
print(f"Map:key value pir  :{mapping}")

"""输出"""
Map:key value pir  :[('scala', 1), ('java', 1), ('hadoop', 1), ('spark', 1), ('akka', 1), ('spark vs hadoop', 1), ('pyspark', 1), ('pyspark and spark', 1)]
```

3.2.6 reduce(f)

- 执行指定的可交换和关联二元操作后，将返回 `RDD` 中的元素。
- 在下面的示例中，我们从运算符导入add包并将其应用于’ `num` ’以执行简单的加法运算。说白了和 `Python` 的 `reduce` 一样：假如有一组整数[x1,x2,x3]，利用 `reduce` 执行加法操作 `add` ，对第一个元素执行 `add` 后，结果为sum=x1,然后再将sum和x2执行 `add` ，sum=x1+x2，最后再将x2和sum执行add，此时sum=x1+x2+x3。

```python
sc.stop()

from pyspark import SparkContext
from operator import add
sc = SparkContext("local", "Reduce app")
nums = sc.parallelize([1, 2, 3, 4, 5])
adding = nums.reduce(add)
print(f"Adding all the elements:{adding}")

"""输出"""
Adding all the elements15
```

3.2.7 join(other, numPartitions = None)

- 它返回 `RDD`，其中包含一对带有匹配键的元素以及该特定键的所有值。

```python
sc.stop()

from pyspark import SparkContext
sc = SparkContext("local", "Join app")
x = sc.parallelize([("spark", 1), ("hadoop", 4)])
y = sc.parallelize([("spark", 2), ("hadoop", 5)])
joined = x.join(y)
final = joined.collect()
print( "Join RDD :%s" % (final))


"""输出"""
Join RDD :[('hadoop', (4, 5)), ('spark', (1, 2))]
```

3.2.8 distinct()

- 去重操作

```python
sc.stop()

from pyspark import SparkConf,SparkContext

sc = SparkContext()

# 创建一个int类型和string类型的数据
intRDD = sc.parallelize([3,1,2,5,5])
stringRDD = sc.parallelize(['Apple','Orange','Grape','Banana','Apple'])

print(intRDD.distinct().collect())
print(stringRDD.distinct().collect())


"""输出"""
[3, 1, 2, 5]
```

```
['Apple', 'Orange', 'Grape', 'Banana']
```

3.2.9 randomSplit()

- 运算将整个集合以随机数的方式按照比例分为多个 `RDD`，比如按照0.4和0.6的比例将intRDD分为两个 `RDD`，并输出

```
sc.stop()

from pyspark import SparkConf,SparkContext

sc = SparkContext()

# 创建一个int类型和string类型的数据
intRDD = sc.parallelize([3,1,2,5,5])
stringRDD = sc.parallelize(['Apple','Orange','Grape','Banana','Apple'])

# x,y = intRDD.randomSplit([0.4,0.6])
# print(x.collect())
# print(y.collect())

sRDD = intRDD.randomSplit([0.4,0.6])
print(len(sRDD))
print(sRDD[0].collect())
print(sRDD[1].collect())


"""输出"""
2
[2, 5]
[3, 1, 5]
```

3.2.10 groupBy()

- `groupBy` 运算可以按照传入匿名函数的规则，将数据分为多个 `Array`。比如下面的代码将intRDD分为偶数和奇数

```
sc.stop()

from pyspark import SparkConf,SparkContext

sc = SparkContext()

# 创建一个int类型和string类型的数据
intRDD = sc.parallelize([3,1,2,5,5])
stringRDD = sc.parallelize(['Apple','Orange','Grape','Banana','Apple'])


```

```
result = intRDD.groupBy(lambda x : x % 2).collect()
print (sorted([(x, sorted(y)) for (x, y) in result]))


"""输出"""
[(0, [2]), (1, [1, 3, 5, 5])]
```

3.3 RDD集合运算

3.3.1 union()

- 进行并集运算

```
sc.stop()
from pyspark import SparkContext

sc = SparkContext()
intRDD1 = sc.parallelize([3,1,2,5,5])
intRDD2 = sc.parallelize([5,6])
intRDD3 = sc.parallelize([2,7])
print (intRDD1.union(intRDD2).union(intRDD3).collect())

"""输出"""

[3, 1, 2, 5, 5, 5, 6, 2, 7]
```

3.3.2 intersection()

- 进行交集运算

```
sc.stop()
from pyspark import SparkContext

sc = SparkContext()
intRDD1 = sc.parallelize([3,1,2,5,5])
intRDD2 = sc.parallelize([5,6])
print (intRDD1.intersection(intRDD2).collect())

"""输出"""
[5]
```

3.3.3 subtract()

- 进行差集运算

```python
sc.stop()
from pyspark import SparkContext

sc = SparkContext()
intRDD1 = sc.parallelize([3,1,2,5,5])
intRDD2 = sc.parallelize([5,6])
print (intRDD1.subtract(intRDD2).collect())

"""输出"""
[2, 3, 1]
```

3.3.4 cartesian()

- 进行笛卡尔乘积运算

```python
sc.stop()
from pyspark import SparkContext

sc = SparkContext()
intRDD1 = sc.parallelize([3,1,2,5,5])
intRDD2 = sc.parallelize([5,6])
print (intRDD1.cartesian(intRDD2).collect())

"""输出"""

[(3, 5), (3, 6), (1, 5), (1, 6), (2, 5), (2, 6), (5, 5), (5, 6),
(5, 5), (5, 6)]
```

由于两个RDD分别有5个元素和2个元素，所有返回结果有10个元素。

3.4 RDD基本动作运算

3.4.1 读取元素

- 读取元素，可以使用下列命令读取 `RDD` 内的元素，这是 `Actions` 运算，所以会马上执行。

```python
sc.stop()

from pyspark import SparkConf,SparkContext

sc = SparkContext()

# 创建一个int类型和string类型的数据
intRDD = sc.parallelize([3,1,2,5,5])
stringRDD =
sc.parallelize(['Apple','Orange','Grape','Banana','Apple'])


#取第一条数据
```

```
print(intRDD.first())
# print(stringRDD.first())
#取前两条数据
print (intRDD.take(2))
# print(stringRDD.take(2))
#升序排列，并取前3条数据
print (intRDD.takeOrdered(3))
# print(stringRDD.takeOrdered(3))
#降序排列，并取前3条数据
print (intRDD.takeOrdered(3,lambda x:-x))


"""输出"""
3
[3, 1]
[1, 2, 3]
[5, 5, 3]
```

--

3.4.2 统计功能

- 可以将 `RDD` 内的元素进行统计运算

```
sc.stop()

from pyspark import SparkConf,SparkContext

sc = SparkContext()

# 创建一个int类型和string类型的数据
intRDD = sc.parallelize([3,1,2,5,5])
stringRDD =
sc.parallelize(['Apple','Orange','Grape','Banana','Apple'])


#统计  会返回一些常见的统计指标的值
print (intRDD.stats())
#最小值
print ('最小值',intRDD.min())
#最大值
print ('最大值',intRDD.max())
#标准差
print ('标准差',intRDD.stdev())
#计数
print ('计数',intRDD.count())
#求和
print ('求和',intRDD.sum())
#平均
print ('平均',intRDD.mean())

"""输出"""
```

```
(count: 5, mean: 3.2, stdev: 1.6, max: 5.0, min: 1.0)
最小值 1
最大值 5
标准差 1.6
计数 5
求和 16
平均 3.2
```

--

3.4.3 单个RDD Key-Value基本“转换”运算

3.4.3.1 key-value 基本运算

- 我们用元素类型为 `tuple` 元组的数组初始化我们的 `RDD` ，这里，每个 `tuple` 的第一个值将作为键，而第二个元素将作为值。
  可以使用 `keys` 和 `values` 函数分别得到 `RDD` 的键数组和值数组：

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
print(kvRDD1.keys().collect())
print(kvRDD1.values().collect())


"""输出"""
[3, 3, 5, 1]
[4, 6, 6, 2]
```

3.4.3.2 键运算

- 筛选元素,可以按照键进行元素筛选，也可以通过值进行元素筛选，和之前的一样，使用filter函数，这里要注意的是，虽然RDD中是以键值对形式存在，但是本质上还是一个二元组，二元组的第一个值代表键，第二个值代表值，所以按照如下的代码既可以按照键进行筛选，我们筛选键的数值小于5的元组：

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
print(kvRDD1.keys().collect())
print(kvRDD1.values().collect())

# 取出键（二元组第一个元素）小于5的二元组
print(kvRDD1.filter(lambda x:x[0]<5).collect())

"""输出"""
```

```
[3, 3, 5, 1]
[4, 6, 6, 2]
[(3, 4), (3, 6), (1, 2)]
```

- 可以使用sortByKey按照key进行排序，传入参数的默认值为true，是按照从小到大排序，也可以传入参数false，表示从大到小排序

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
print(kvRDD1.keys().collect())
print(kvRDD1.values().collect())

# 取出键（二元组第一个元素）小于5的二元组
print(kvRDD1.sortByKey(False).collect())
print(kvRDD1.sortByKey(True).collect())

"""输出"""
[3, 3, 5, 1]
[4, 6, 6, 2]
[(5, 6), (3, 4), (3, 6), (1, 2)]
[(1, 2), (3, 4), (3, 6), (5, 6)]
```

3.4.3.3 值运算

- 值运算，我们可以使用mapValues()方法处理value值，下面的代码将value值进行了平方处理

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
# print(kvRDD1.keys().collect())
# print(kvRDD1.values().collect())

# 对值进行平方
print(kvRDD1.mapValues(lambda x:x**2).collect())


"""输出"""
[(3, 16), (3, 36), (5, 36), (1, 4)]
```

- 使用reduceByKey函数可以对具有相同key值的数据进行合并。比如下面的代码，由于RDD中存在（3,4）(3,10)和（3,6）3条key值均为3的数据，他们将被合为一条数据。

```python
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2),(3,10)])
print(kvRDD1.keys().collect())
print(kvRDD1.values().collect())

# 对具有相同key值的数据进行合并
print(kvRDD1.reduceByKey(lambda x,y:x+y).collect())

"""输出"""
[3, 3, 5, 1, 3]
[4, 6, 6, 2, 10]
[(3, 20), (5, 6), (1, 2)]
```

3.4.4 多个RDD Key-Value“转换”运算

3.4.4.1 key-value “动作”运算

- key-value “动作”运算

```python
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])


# Key-Value“动作”运算
#读取第一条数据
print (kvRDD1.first())
#读取前两条数据
print (kvRDD1.take(2))
#读取第一条数据的key值
print (kvRDD1.first()[0])
#读取第一条数据的value值
print (kvRDD1.first()[1])

"""输出"""
(3, 4)
[(3, 4), (3, 6)]
3
4
```

3.4.4.2 连接语句

- 在对多个rdd进行计算时，首先考虑使用连接语句，将多个rdd转化为一个rdd，并且为了保证数据的不丢失，要恰当地选择合适的连接方法。下面将进行常见的连接方法的介绍。
    1. 内连接
    2. 左外连接
    3. 右外连接
- 内连接运算，join运算可以实现类似数据库的内连接，将两个RDD按照相同的key值join起来，kvRDD1与kvRDD2的key值唯一相同的是3，kvRDD1中有两条key值为3的数据(3,4)和(3,6)，而kvRDD2中只有一条key值为3的数据(3,8)，所以join的结果是(3，(4,8))和(3，(6，8))，对于没有相同key的数据，将不再进行显示：

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

# 内连接运算
print (kvRDD1.join(kvRDD2).collect())

"""输出"""
[(3, (4, 8)), (3, (6, 8))]
```

- 左外连接运算，使用leftOuterJoin可以实现类似数据库的左外连接，如果kvRDD1的key值对应不到kvRDD2，就会显示None，这是与内连接不同的一点。

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

# 左外连接运算
print (kvRDD1.leftOuterJoin(kvRDD2).collect())


"""输出"""
[(3, (4, 8)), (3, (6, 8)), (5, (6, None)), (1, (2, None))]
```

- 右外连接运算，使用rightOuterJoin可以实现类似数据库的右外连接，如果kvRDD2的key值对应不到kvRDD1，就会显示None

```python
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

# 右外连接运算
print (kvRDD1.rightOuterJoin(kvRDD2).collect())

"""输出"""
[(3, (4, 8)), (3, (6, 8))]
```

3.4.4.3 键运算

- 删除相同key值数据，使用subtractByKey运算会删除相同key值得数据：

```python
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

print (kvRDD1.subtractByKey(kvRDD2).collect())

"""输出"""
[(5, 6), (1, 2)]
```

- 按key值统计，使用countByKey函数可以统计各个key值对应的数据的条数：

```python
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

print (kvRDD1.countByKey())

"""输出"""
defaultdict(<class 'int'>, {3: 2, 5: 1, 1: 1})
```

- lookup查找运算，使用lookup函数可以根据输入的key值来查找对应的Value值：

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

print(kvRDD1.lookup(3))

"""输出"""
[4, 6]
```

3.4.4.4 值运算

- 按value值统计，使用countByValue函数可以统计各个key值对应的数据的条数，注意与countByKey输出的区别：

```
sc.stop()

from pyspark import SparkContext

sc = SparkContext()
kvRDD1 = sc.parallelize([(3,4),(3,6),(5,6),(1,2)])
kvRDD2 = sc.parallelize([(3,8)])

print (kvRDD1.countByValue())

"""输出"""
defaultdict(<class 'int'>, {(3, 4): 1, (3, 6): 1, (5, 6): 1, (1,
2): 1})
```

3.4.5 持久化操作

- spark RDD的持久化机制，可以将需要重复运算的RDD存储在内存中，以便大幅提升运算效率，有两个主要的函数：
    1. persist() 使用persist函数对RDD进行持久化
    2. unpersist() 使用unpersist函数对RDD进行去持久化

---

4 三、DataFrame的基本操作

- 在 `pyspark` 中除了 `rdd` 之外还有 `dataframe` 这种数据格式。他们之间的差别在于：dataframe比rdd的速度快，对于结构化的数据，使用dataframe编写的代码更简洁。对于非结构化数据，建议先使用rdd处理成结构化数据，然后转换成dataframe。在dataFrame中的操作主要可以分为查、增改、合并、统计、删除、去重、格式转换、SQL操作、读写csv文件几个大的方向。我们从数据库中查询到的数据返回的数据结构就是DataFrame格式的。
- 首先加载数据

```
sc.stop()
from pyspark import SparkContext
from pyspark.sql import SQLContext
sc = SparkContext()
sqlContext = SQLContext(sc)
df = sqlContext.read.format('com.databricks.spark.csv').options(header='true', inferschema='true').load(r'./数据/小微企业用电量数据.csv')
```

4.1 查的操作

4.1.1 行元素查询操作

4.1.1.1 show()

- 显示前5行数据,会逐行显示出前5行数据，包含表头

```
df.show(5)
```

```
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|    区域|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|
year|month|day|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|2018-03-12|2018|  3|   1|      11|   0| 12|兰州市|  皋兰县|null|null|   1291.71|          16|           2|           7
|           7|2018|    3| 12|
|2016-10-26|2016| 10|   4|      43|   2| 26|兰州市|  西固区|null|null|   4154.02|          15|           0|           3
|          12|2016|   10| 26|
|2016-06-18|2016|  6|   2|      24|   5| 18|兰州市|  榆中县|null|null|   4040.13|          28|           5|          13
|          10|2016|    6| 18|
|2018-06-30|2018|  6|   2|      26|   5| 30|兰州市|七里河区|null|null|   51109.7|         124|           3|          31
|          90|2018|    6| 30|
|2016-01-10|2016|  1|   1|       1|   6| 10|兰州市|  安宁区|null|null|    1353.5|           7|           0|           2
|           5|2016|    1| 10|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
only showing top 5 rows
```

4.1.1.2 printSchema()

- 以树的形式打印概要

```
#以树的形式打印概要
df.printSchema()
```

```
root
 |-- 日期: string (nullable = true)
 |-- 年: integer (nullable = true)
 |-- 月: integer (nullable = true)
 |-- 季度: integer (nullable = true)
 |-- 周(年度): integer (nullable = true)
 |-- 星期: integer (nullable = true)
 |-- 日: integer (nullable = true)
 |-- 城市: string (nullable = true)
 |-- 区域: string (nullable = true)
 |-- 分类: string (nullable = true)
 |-- 等级: string (nullable = true)
 |-- 日冻结电量: double (nullable = true)
 |-- 小微企业数量: integer (nullable = true)
 |-- 第一产业数量: integer (nullable = true)
 |-- 第二产业数量: integer (nullable = true)
 |-- 第三产业数量: integer (nullable = true)
 |-- year: integer (nullable = true)
 |-- month: integer (nullable = true)
 |-- day: integer (nullable = true)
```

4.1.1.3 dtypes

- 查看数据类型

```python
# 查看数据类型
df.dtypes
```

```
[('日期', 'string'),
 ('年', 'int'),
 ('月', 'int'),
 ('季度', 'int'),
 ('周(年度)', 'int'),
 ('星期', 'int'),
 ('日', 'int'),
 ('城市', 'string'),
 ('区域', 'string'),
 ('分类', 'string'),
 ('等级', 'string'),
 ('日冻结电量', 'double'),
 ('小微企业数量', 'int'),
 ('第一产业数量', 'int'),
 ('第二产业数量', 'int'),
 ('第三产业数量', 'int'),
 ('year', 'int'),
 ('month', 'int'),
 ('day', 'int')]
```

4.1.1.4 head()

- 获取头几行数据，以python中的列表形式返回

```
list1 = df.head(3)
print(type(list1))
print(list1)

"""输出"""
<class 'list'>
[Row(日期='2018-03-12', 年=2018, 月=3, 季度=1, 周(年度)=11, 星期=0, 日=12, 城市='兰州市', 区域='皋兰县', 分类=None, 等级=None, 日冻结电量=1291.71, 小微企业数量=16, 第一产业数量=2, 第二产业数量=7, 第三产业数量=7, year=2018, month=3, day=12), Row(日期='2016-10-26', 年=2016, 月=10, 季度=4, 周(年度)=43, 星期=2, 日=26, 城市='兰州市', 区域='西固区', 分类=None, 等级=None, 日冻结电量=4154.02, 小微企业数量=15, 第一产业数量=0, 第二产业数量=3, 第三产业数量=12, year=2016, month=10, day=26), Row(日期='2016-06-18', 年=2016, 月=6, 季度=2, 周(年度)=24, 星期=5, 日=18, 城市='兰州市', 区域='榆中县', 分类=None, 等级=None, 日冻结电量=4040.13, 小微企业数量=28, 第一产业数量=5, 第二产业数量=13, 第三产业数量=10, year=2016, month=6, day=18)]

```

4.1.1.5 count()

- 查询总行数

```
int_num = df.count()
int_num

"""输出"""
14481
```

4.1.1.6 alias()

- 取别名

```
# 取别名
df.select(df.日期.alias("日期别名")).show(2)


"""输出"""
+----------+
|  日期别名|
+----------+
|2018-03-12|
|2016-10-26|
+----------+
only showing top 2 rows
```

4.1.1.7 psf.isnull()

- 查询某列为null的行

```
mport pyspark.sql.functions as psf

#查询某列为null的行
df_null = df.filter(psf.isnull("分类"))
df_null.show(2)
```

```
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|  区域|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|ye
ar|month|day|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|2018-03-12|2018|  3|   1|      11|   0| 12|兰州市|皋兰县|null|null|   1291.71|          16|           2|           7|
7|2018|    3| 12|
|2016-10-26|2016| 10|   4|      43|   2| 26|兰州市|西固区|null|null|   4154.02|          15|           0|           3|
12|2016|   10| 26|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
only showing top 2 rows
```

4.1.1.8 collect()

- 将数据以python列表的格式输出

```
df_list = df.collect() #将数据以python的列表格式输出
print(type(df_list))
df_list
```

```
<class 'list'>

1[Row(日期='2018-03-12', 年=2018, 月=3, 季度=1, 周(年度)=11, 星期=0, 日=12, 城市='兰州市', 区域='皋兰县', 分类=None, 等级=Non
e, 日冻结电量=1291.71, 小微企业数量=16, 第一产业数量=2, 第二产业数量=7, 第三产业数量=7, year=2018, month=3, day=12),
2Row(日期='2016-10-26', 年=2016, 月=10, 季度=4, 周(年度)=43, 星期=2, 日=26, 城市='兰州市', 区域='西固区', 分类=None, 等级=Non
e, 日冻结电量=4154.02, 小微企业数量=15, 第一产业数量=0, 第二产业数量=3, 第三产业数量=12, year=2016, month=10, day=26),
3Row(日期='2016-06-18', 年=2016, 月=6, 季度=2, 周(年度)=24, 星期=5, 日=18, 城市='兰州市', 区域='榆中县', 分类=None, 等级=Non
e, 日冻结电量=4040.13, 小微企业数量=28, 第一产业数量=5, 第二产业数量=13, 第三产业数量=10, year=2016, month=6, day=18),
 Row(日期='2018-06-30', 年=2018, 月=6, 季度=2, 周(年度)=26, 星期=5, 日=30, 城市='兰州市', 区域='七里河区', 分类=None, 等级=Non
e, 日冻结电量=51109.7, 小微企业数量=124, 第一产业数量=3, 第二产业数量=31, 第三产业数量=90, year=2018, month=6, day=30),
 Row(日期='2016-01-10', 年=2016, 月=1, 季度=1, 周(年度)=1, 星期=6, 日=10, 城市='兰州市', 区域='安宁区', 分类=None, 等级=None,
日冻结电量=1353.5, 小微企业数量=7, 第一产业数量=0, 第二产业数量=2, 第三产业数量=5, year=2016, month=1, day=10),
 Row(日期='2017-07-28', 年=2017, 月=7, 季度=3, 周(年度)=30, 星期=4, 日=28, 城市='兰州市', 区域='红古区', 分类=None, 等级=Non
e, 日冻结电量=1428.25, 小微企业数量=18, 第一产业数量=2, 第二产业数量=5, 第三产业数量=11, year=2017, month=7, day=28),
 Row(日期='2018-01-25', 年=2018, 月=1, 季度=1, 周(年度)=4, 星期=3, 日=25, 城市='兰州市', 区域='永登县', 分类='道路结冰', 等级
='黄', 日冻结电量=10949.11, 小微企业数量=100, 第一产业数量=24, 第二产业数量=36, 第三产业数量=40, year=2018, month=1, day=25),
 Row(日期='2019-12-28', 年=2019, 月=12, 季度=4, 周(年度)=52, 星期=5, 日=28, 城市='兰州市', 区域='永登县', 分类=None, 等级=Non
```

4.1.1.9 describe()

- 对df中的数据进行统计，返回常用的一些统计指标的值

```
df["小微企业数量","第三产业数量"].describe().show()

"""输出"""
+-------+------------------+------------------+
|summary|       小微企业数量|       第三产业数量|
+-------+------------------+------------------+
|  count|             14481|             14481|
|   mean|105.62433533595747| 70.57074787652786|
| stddev| 137.9081201821359|115.61275253002611|
|    min|                 3|                 2|
|    max|              1098|               932|
+-------+------------------+------------------+
```

4.1.1.10 distinct()

- 去重

```
before_dist = df.select("分类","等级").count()
after_dist = df.select("分类","等级").distinct().count()
print('去重之前',before_dist)
print('去重之后',after_dist)

df.select("分类","等级").distinct().show()

"""输出"""
去重之前 14481
去重之后 18
+--------+----+
|    分类|等级|
+--------+----+
|    寒潮|  蓝|
|    暴雨|  橙|
|    暴雨|  蓝|
|    null|null|
|    暴雨|  黄|
|    大风|  蓝|
|    霜冻|  蓝|
|    雷电|  橙|
|    霜冻|  黄|
|    高温|  红|
|    暴雨|  红|
|    雷电|  黄|
|    冰雹|  橙|
|道路结冰|  黄|
|    高温|  橙|
|  沙尘暴|  黄|
|    高温|  黄|
|雷雨大风|  黄|
+--------+----+
```

4.1.1.11 sample()

- 随机取样

```
sample = df.sample(withReplacement=False
                   ,fraction=0.5
                   ,seed=0)
sample.show(2) #随机取样，抽取50%的数据出来
```

```
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|     区域|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|
year|month|day|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|2016-06-18|2016|  6|   2|      24|   5| 18|兰州市|  榆中县|null|null|   4040.13|          28|           5|          13
|          10|2016|    6| 18|
|2018-06-30|2018|  6|   2|      26|   5| 30|兰州市|七里河区|null|null|   51109.7|         124|           3|          31
|          90|2018|    6| 30|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
only showing top 2 rows
```

4.1.1.12 expr()

- 将字符串作为一个函数来执行

```python
# expr() 将字符串作为一个函数来执行
import pyspark.sql.functions as psf
df_tran = df.select(df.年.alias('year'))
df_tran.select(psf.expr("length(year)")).show(5) # 返回' year '列的长度


"""输出"""
+----------------------------+
|length(CAST(year AS STRING))|
+----------------------------+
|                           4|
|                           4|
|                           4|
|                           4|
|                           4|
+----------------------------+
only showing top 5 rows
```

4.1.2 列元素查询

4.1.2.1 columns

- 获取df中的列名，注意columns后面没有括号

```python
df.columns

"""输出"""

['日期', '年', '月', '季度', '周(年度)', '星期', '日', '城市', '区域', '分类', '等级', '日冻结电量', '小微企业数量', '第一产业数量', '第二产业数量', '第三产业数量', 'year', 'month', 'day']
```

4.1.2.2 select

- 选取某一列或某几列数据

```
#使用select返回的是dataframe格式,
print(type(df.select('年')))
df.select("年").show(2)

"""输出"""
<class 'pyspark.sql.dataframe.DataFrame'>
+----+
|  年|
+----+
|2018|
|2016|
+----+
only showing top 2 rows
```

- 使用df[]在选中>=2个列时返回的才是dataframe对象，否则返回的是column对象。

```
print(type(df['年']))
print(type(df[['年','月']]))



<class 'pyspark.sql.column.Column'>
<class 'pyspark.sql.dataframe.DataFrame'>
```

- 选择多列

```
df.select('年','月','日').show(2)
或
df.select(df.年,df.月,df.日).show(2)



+----+---+---+
|  年| 月| 日|
+----+---+---+
|2018|  3| 12|
|2016| 10| 26|
+----+---+---+
only showing top 2 rows
```

- 选择满足条件的数据，注意在字符串上要加引号,以下三种写法等价

```
# 选择满足条件的数据，注意在字符串上要加引号,以下三种写法等价
df.where('year=2016 and month=1').show(2)
df.where(df.year==2016).where(df.month==1).show(2)
df.where((df.year==2016)&(df.month==1)).show(2)
```

```
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|  区域|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|ye
ar|month|day|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|2016-01-10|2016|  1|   1|       1|   6| 10|兰州市|安宁区|null|null|    1353.5|           7|           0|           2|
5|2016|    1| 10|
|2016-01-13|2016|  1|   1|       2|   2| 13|兰州市|榆中县|null|null|   2402.32|          18|           5|           9|
4|2016|    1| 13|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
only showing top 2 rows
```

4.1.2.3 withColumnRenamed()

- 重命名列 `withColumnRenamed` (“old_name”,”new_name”)

  withColumnRenamed不可直接在原数据上修改

```
df = df.withColumnRenamed("区域",'area')
df.columns
```

```
['日期', '年', '月', '季度', '周(年度)', '星期', '日', '城市', 'area', '分类', '等级', '日冻结电量', '小微企业数量', '第一产业数量', '第二产业数量', '第三产业数量', 'year', 'month', 'day']
```

4.1.2.4 like(expression)

- 判断列中的值是否满足相似条件,判断条件跟sql语法相同,支持通配符

```
df.select(df.year.like('201%').alias('like_year')).show(2)



+---------+
|like_year|
+---------+
|     true|
|     true|
+---------+
only showing top 2 rows
```

4.1.2.5 orderBy()

- 排序，可以指定正向排序或者倒序排列

```
df.select("year").orderBy("year",ascending=False).show(5)

+----+
|year|
+----+
|2016|
|2016|
|2016|
|2016|
|2016|
+----+
only showing top 5 rows
```

- 对df所有列按年龄排序(ascending=False 或者True desc()换成asc()可控制升降序)

```
df.orderBy('year',ascending=False).show(5)
#或者
df.orderBy(df.year.desc()).show(5)
#或者
df.sort('year',ascending=False).show(5)
#或者
df.sort(df.year.desc()).show(5)
```

4.1.2.6 窗口函数Window

- 使用窗口函数进行对比：

```
from pyspark.sql import functions as psf
from pyspark.sql.window import Window
df.select("year",psf.rank().over(Window.orderBy("year")).alias("year_rank") ).distinct().show(5)
df.select("year",psf.rank().over(Window.orderBy(df.year.desc())).alias("year_rank") ).distinct().show(5)
df.orderBy(df.year.desc()).show(5)



"""输出"""
+----+---------+
|year|year_rank|
+----+---------+
|2016|        1|
|2017|     3295|
|2018|     6580|
|2019|     9847|
|2020|    13123|
+----+---------+

+----+---------+
|year|year_rank|
```

```
+----+---------+
|2020|        1|
|2019|     1360|
|2018|     4636|
|2017|     7903|
|2016|    11188|
+----+---------+
```

```
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|     area|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数
量|year|month|day|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|2020-04-07|2020|  4|   2|      15|   1|  7|兰州市|  永登县|null|null|  23805.75|         204|          40|          60
|         104|2020|   4|  7|
|2020-04-05|2020|  4|   2|      14|   6|  5|兰州市|七里河区|null|null| 166898.36|         373|           7|          83
|         283|2020|   4|  5|
|2020-01-02|2020|  1|   1|       1|   3|  2|兰州市|  榆中县|null|null| 120044.28|         212|          41|          65
|         106|2020|   1|  2|
|2020-05-21|2020|  5|   2|      21|   3| 21|兰州市|  安宁区|null|null|  30183.27|         202|           7|          69
|         126|2020|   5| 21|
|2020-01-21|2020|  1|   1|       4|   1| 21|兰州市|七里河区|null|null| 158494.59|         344|           7|          75
|         262|2020|   1| 21|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
only showing top 5 rows
```

4.1.2.7 sample()

- sample()

```python
#sample() #随机抽样
t1 = df.sample(False, 0.01, 3)
print(len(t1.collect()))
t2 = df.sample(False, 0.1, 3)
print(len(t2.collect()))


"""输出"""
154
1403
```

4.1.2.8 withReplacement

- withReplacement = True or False代表是否在原数据进行修改。

4.1.2.9 when、others

- when(condition, value1).otherwise(value2)，意为：当满足条件condition的值时赋值为values1,不满足条件的则赋值为values2，otherwise表示，不满足条件的情况下，应该赋值何值。

```python
from pyspark.sql import functions as psf

df.select(df.日期
          ,psf.when(df.year=='2016',1)\
          .when(df.year=='2017',2)\
          .when(df.year=='2018',3)\
```

```
            .when(df.year=='2019',4)\
            .otherwise(5).alias('year_rank'))\
.show(10)

"""输出"""
+----------+---------+
|      日期|year_rank|
+----------+---------+
|2018-03-12|        3|
|2016-10-26|        1|
|2016-06-18|        1|
|2018-06-30|        3|
|2016-01-10|        1|
|2017-07-28|        2|
|2018-01-25|        3|
|2019-12-28|        4|
|2017-09-25|        2|
|2017-08-19|        2|
+----------+---------+
only showing top 10 rows
```

4.1.2.10 between()、psf.monotonically_increasing_id()

- between(lowerBound, upperBound) 筛选出某个范围内的值，返回的是TRUE or FALSE

```
df.select(
    df.日期
    ,df.year.between(2016,2017).alias('2016-2017')
).show(5)

"""输出"""
+----------+---------+
|      日期|2016-2017|
+----------+---------+
|2018-03-12|    false|
|2016-10-26|     true|
|2016-06-18|     true|
|2018-06-30|    false|
|2016-01-10|     true|
+----------+---------+
only showing top 5 rows
```

- 如果需要进行筛选df中的特定区间内的数据时，可以使用下面这种通过行索引进行筛选的方式。

```
dfWithIndex = df.withColumn("索引",psf.monotonically_increasing_id())  #为df添加行索引
dfWithIndex.select(dfWithIndex.索引
                   ,dfWithIndex.日期
                   ,dfWithIndex.年.between(2016,2017).alias('2016-2017')
                  ).show(4)
```

```

"""输出"""
+----+----------+---------+
|索引|      日期|2016-2017|
+----+----------+---------+
|   0|2018-03-12|    false|
|   1|2016-10-26|     true|
|   2|2016-06-18|     true|
|   3|2018-06-30|    false|
+----+----------+---------+
only showing top 4 rows
```

4.1.2.11 filter()

- filter() #过滤数据

```
df.filter(df['year']>=2018).show(2)
等价于
df.where("year >= 2018").show(2)
等价于
df.where(df.year>=2018).show(2)
等价于
df.where(df['year']>=2018).show(2)
```

```
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|    area|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数
量|year|month|day|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
|2018-03-12|2018|  3|  1|      11|   0| 12|兰州市|  皋兰县|null|null|   1291.71|          16|           2|           7
|           7|2018|    3| 12|
|2018-06-30|2018|  6|  2|      26|   5| 30|兰州市|七里河区|null|null|   51109.7|         124|           3|          31
|          90|2018|    6| 30|
+----------+----+---+----+--------+----+---+------+--------+----+----+----------+------------+------------+----------
--+------------+----+-----+---+
only showing top 2 rows
```

4.1.2.12 psf.isnull、psf.isnan

- psf.isnull()

```
# 把a列里面数据为null的筛选出来（代表python的None类型）
df.filter(psf.isnull("等级")).show(2)
```

```
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|  area|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|y
ear|month|day|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
|2018-03-12|2018|  3|  1|      11|   0| 12|兰州市|皋兰县|null|null|   1291.71|          16|           2|           7|
7|2018|    3| 12|
|2016-10-26|2016| 10|  4|      43|   2| 26|兰州市|西固区|null|null|   4154.02|          15|           0|           3|
12|2016|   10| 26|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------
+------------+----+-----+---+
only showing top 2 rows
```

- psf.isnan()

```
# 把a列里面数据为nan的筛选出来（Not a Number，非数字数据）
df.filter(psf.isnan("等级")).show(2)
```

```
+----+---+---+----+--------+----+---+----+----+----+----+----------+------------+------------+------------+----------
--+----+-----+---+
|日期| 年| 月|季度|周(年度)|星期| 日|城市|area|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|year|month|d
ay|
+----+---+---+----+--------+----+---+----+----+----+----+----------+------------+------------+------------+----------
--+----+-----+---+
+----+---+---+----+--------+----+---+----+----+----+----+----------+------------+------------+------------+----------
--+----+-----+---+
```

4.2 增改的操作

4.2.1 新建、新增数据

1. 第一种，将pandas中的DataFrame转为spark中的DataFrame
2. 第二种，将rdd转为spark中的DataFrame格式
3. 第三种，将spark中的DataFrame格式转为pandas中的DataFrame，使用toPandas()
4. 第四种，withColumn() 在df中新增数据列，会返回一个新的DataFrame，需要两个参数，

4.2.1.1 pd.DF –> spark.DF

- 将pandas中的DataFrame转为spark中的DataFrame

```
# 第一种，将pandas中的DataFrame转为spark中的DataFrame

import pandas as pd
from pyspark.sql import SparkSession

#初始化spark会话
spark = SparkSession.builder.getOrCreate()
pandas_df = pd.DataFrame({"name":["ss","aa","qq","ee"],"age":
[12,18,20,25]})
print('type of pandas_df:',type(pandas_df))
#将pandas中的DataFrame转化为pyspark中的DataFrame
spark_df = spark.createDataFrame(pandas_df)
print('type of spark_df:',type(spark_df))
spark_df.show()

"""输出"""
type of pandas_df: <class 'pandas.core.frame.DataFrame'>
type of spark_df: <class 'pyspark.sql.dataframe.DataFrame'>
+----+---+
|name|age|
+----+---+
|  ss| 12|
|  aa| 18|
|  qq| 20|
```

```
|  ee| 25|
+----+---+
```

4.2.1.2 RDD –>spark.DF

- 将rdd转为spark中的DataFrame格式

```
sc.stop()

# 第二种，将rdd转为spark中的DataFrame格式
from pyspark.sql import Row
from pyspark import SparkContext
sc = SparkContext()
row = Row('spe_id','InOther')
x = ['x1','x2']
y = ['y1','y2']
new_df = sc.parallelize([row(x[i],y[i]) for i in range(2)])
print('new_df:',new_df.collect())
print('type of new_df:',type(new_df))
ss = new_df.toDF()
print('type of ss:',type(ss))

"""输出"""
new_df: [Row(spe_id='x1', InOther='y1'), Row(spe_id='x2', InOther='y2')]
type of new_df: <class 'pyspark.rdd.RDD'>
type of ss: <class 'pyspark.sql.dataframe.DataFrame'>
```

4.2.1.3 spark.DF ->pd.DF

- 将spark中的DataFrame格式转为pandas中的DataFrame

```
# 第三种，将spark中的DataFrame格式转为pandas中的DataFrame，使用toPandas()
import pandas as pd
from pyspark.sql import SparkSession
# 初始化spark会话
spark = spark.builder.getOrCreate()
pandas_df = pd.DataFrame({"name":["ss","aa","qq","ee"],"age":[12,18,20,25]})
#将pandas中的DataFrame转化为pyspark中的DataFrame
spark_df = spark.createDataFrame(pandas_df)
print('type of spark_df:',type(spark_df))
new_pandas = spark_df.toPandas()
print('type of new_pandas:',type(new_pandas))


"""输出"""
type of spark_df: <class 'pyspark.sql.dataframe.DataFrame'>
type of new_pandas: <class 'pandas.core.frame.DataFrame'>
```

4.2.1.4 withColumn

- withColumn() 在df中新增数据列，会返回一个新的DataFrame，需要两个参数，第二个参数是对原来的列作何操作，比如原来的列整体加一，不可以从别的列进行取数。

```
df1 = df.select("year",'month')
withcolumns_df = df1.withColumn('month+1',df1.month+1)
withcolumns_df.show(5)

"""输出"""
+----+-----+-------+
|year|month|month+1|
+----+-----+-------+
|2018|    3|      4|
|2016|   10|     11|
|2016|    6|      7|
|2018|    6|      7|
|2016|    1|      2|
+----+-----+-------+
only showing top 5 rows
```

4.2.1.5 cast

- 修改列的类型

```
# 修改列的类型
cast_before = df.select('year')
cast_before.dtypes
cast_after = df.withColumn("year",df['year'].cast("String")).select('year')
print(f'修改前 year 列 类型: {cast_before.dtypes}')
print(f'修改后 year 列 类型: {cast_after.dtypes}')

"""输出"""
修改前 year 列 类型: [('year', 'int')]
修改后 year 列 类型: [('year', 'string')]
```

4.2.1.6 lit

- lit

```
import time
current_time = time.strftime('%Y-%m-%d %T')

df2 = df.select("year",'month')
withcolumns_df = df2.withColumn('curtime',psf.lit(current_time))
withcolumns_df.show(5)

"""输出"""
+----+-----+-------------------+
|year|month|            curtime|
+----+-----+-------------------+
```

```
|2018|     3|2021-12-15 23:53:01|
|2016|    10|2021-12-15 23:53:01|
|2016|     6|2021-12-15 23:53:01|
|2018|     6|2021-12-15 23:53:01|
|2016|     1|2021-12-15 23:53:01|
+----+-----+-------------------+
only showing top 5 rows
```

4.2.1.7 withColumnRenamed

- 修改列名

```
print(df.withColumnRenamed('year','year_after').columns)

"""输出"""
['日期', '年', '月', '季度', '周(年度)', '星期', '日', '城市', '区域', '分类', '等级', '日冻结电量', '小微企业数量', '第一产业数量', '第二产业数量', '第三产业数量', 'year_after', 'month', 'day']
```

4.3 合并、集合的操作

4.3.1 union()

- 横向拼接，要求合并的两端具有相同的列名及列数

```
# union() #横向拼接，要求合并的两端具有相同的列名及列数
df2 = df.select('年','月')
df3 = df.select(df.year.alias('年'),df.month.alias("月"))
df4 = df2.union(df3) ##等价于unionAll ()
df4.count()


"""输出"""
28962
```

4.3.2 join()

- 根据条件进行关联

```
# Join() #根据条件进行关联
# 单字段关联
#其中，方法可以为：inner, outer, left_outer, right_outer, leftsemi.
#注意，一般使用left_outer进行关联,如果主键不唯一就会造成笛卡尔积，比如下面这种。
df5 = df.select('年','月')
df6 = df.select('月','日')
df7 = df5.join(df6,df5.月==df6.月,'left_outer')
df7.show(5)
df7.count()
```

```

"""输出"""
+----+---+---+---+
|  年| 月| 月| 日|
+----+---+---+---+
|2018|  3|  3| 11|
|2018|  3|  3| 21|
|2018|  3|  3| 17|
|2018|  3|  3| 26|
|2018|  3|  3| 12|
+----+---+---+---+
only showing top 5 rows
```

4.3.3 subtract()、intersect()、union()

- 构建两个dataframe

```
from pyspark.sql.session import SparkSession
spark = SparkSession.builder.getOrCreate()

#求并集、交集
#实例，构建两个dataframe
sentenceDataFrame = spark.createDataFrame((
      (1, "asf"),
      (2, "2143"),
      (3, "rfds")
    )).toDF("label", "sentence")
sentenceDataFrame.show()

sentenceDataFrame1 = spark.createDataFrame((
      (1, "asf"),
      (2, "2143"),
      (4, "f8934y")
    )).toDF("label", "sentence")
sentenceDataFrame1.show()

"""输出"""
+-----+--------+
|label|sentence|
+-----+--------+
|    1|     asf|
|    2|    2143|
|    3|    rfds|
+-----+--------+

+-----+--------+
|label|sentence|
+-----+--------+
|    1|     asf|
|    2|    2143|
|    4|  f8934y|
+-----+--------+
```

4.3.3.1 subtract()

- 求差集，可指定对某一列求差集，如不指定，将对所有列求差集

```
newDF = sentenceDataFrame1.select('sentence')\
        .subtract(sentenceDataFrame.select('sentence'))
newDF.show()


"""输出"""
+--------+
|sentence|
+--------+
|  f8934y|
+--------+
```

4.3.3.2 intersect()

- 交集，可以指定对某一列求交集，如不指定，将对所有列求交集

```
newDF2 = sentenceDataFrame1.select('sentence')\
        .intersect(sentenceDataFrame.select('sentence'))
newDF2.show()

"""输出"""
+--------+
|sentence|
+--------+
|     asf|
|    2143|
+--------+
```

4.3.3.3 union()

- 并集，与上面的表连接的语句相同

```
newDF3 = sentenceDataFrame1.select('sentence')\
        .union(sentenceDataFrame.select('sentence'))
newDF3.show()

"""输出"""
+--------+
|sentence|
+--------+
|     asf|
|    2143|
|  f8934y|
|     asf|
|    2143|
|    rfds|
+--------+
```

4.3.3.4 union+distinct

- 求并集并进行去重，结果可与上面union对比

```
#union+distinct #求并集并进行去重，结果可与上图union对比
newDF4 = sentenceDataFrame1.select('sentence')\
        .union(sentenceDataFrame.select('sentence')).distinct()
newDF4.show()

"""输出"""
+--------+
|sentence|
+--------+
|    rfds|
|     asf|
|    2143|
|  f8934y|
+--------+
```

4.4 explode()与split()

- 对某列的数据以固定的分隔符进行分隔，并新增一列

```
# explode()与split() #对某列的数据以固定的分隔符进行分隔，并新增一列
from pyspark.sql import functions as psf
df11 = df.select('日期')
df11 = df11.withColumn('日期_explod',psf.explode(psf.split(df11.日期,'-')))
df11.show(9)

# 备注，对于分隔符不同意的则转换出来的也不同

"""输出"""
+----------+-----------+
|      日期|日期_explod|
+----------+-----------+
|2018-03-12|       2018|
|2018-03-12|         03|
|2018-03-12|         12|
|2016-10-26|       2016|
|2016-10-26|         10|
|2016-10-26|         26|
|2016-06-18|       2016|
|2016-06-18|         06|
|2016-06-18|         18|
+----------+-----------+
only showing top 9 rows
```

4.5 统计的操作

4.5.1 df.stat.freqItems()

- 频数统计与筛选

```
# 频数统计与筛选，根据 年 与 城市 字段，统计该字段值出现频率在40%以上的内容
df.stat.freqItems(["年",'区域'], 0.4).show()

"""输出"""
+------------+--------------+
|年_freqItems|区域_freqItems|
+------------+--------------+
|[2018, 2016]|            []|
+------------+--------------+
```

4.5.2 crosstab()

- 交叉分析

```
# crosstab() #交叉分析
df.crosstab('分类','等级').show()

"""输出"""
+---------+-----+---+---+---+---+
|分类_等级| null| 橙| 红| 蓝| 黄|
+---------+-----+---+---+---+---+
|     null|12159|  0|  0|  0|  0|
| 雷雨大风|    0|  0|  0|  0|  9|
|   沙尘暴|    0|  0|  0|  0|  9|
|     雷电|    0| 18|  0|  0|954|
|     高温|    0|144| 27|  0|117|
|     冰雹|    0| 36|  0|  0|  0|
|     寒潮|    0|  0|  0| 45|  0|
|     大风|    0|  0|  0|180|  0|
| 道路结冰|    0|  0|  0|  0|513|
|     霜冻|    0|  0|  0|153|  9|
|     暴雨|    0| 18| 18| 63|  9|
+---------+-----+---+---+---+---+
```

4.5.3 groupby()

- 分组函数

```
#groupBy() #分组函数：分组并平均
df.groupby('年').agg({'日冻结电量':'mean'}).show()


#另一个使用实例：分组并计数
df.groupby('年').count().show()
```

```

"""输出"""
+----+------------------+
|  年|    avg(日冻结电量)|
+----+------------------+
|2018|32594.561900826477|
|2019| 47396.74864682531|
|2020| 71951.65986754964|
|2016| 5774.134211900422|
|2017| 18288.82305388127|
+----+------------------+

+----+-----+
|  年|count|
+----+-----+
|2018| 3267|
|2019| 3276|
|2020| 1359|
|2016| 3294|
|2017| 3285|
+----+-----+
```

- 在分组后使用多个函数进行统计，返回的是DataFrame类型

```
# 在分组后使用多个函数进行统计，返回的是DataFrame类型
df.groupby('年').agg(
    psf.avg('日冻结电量')
    ,psf.min('日冻结电量')
    ,psf.max('日冻结电量')
).show()

# 与下面的写法区分，两种不是同一种写法
df.groupBy('年').agg({'日冻结电量':'mean','日冻结电量':'min','日冻结电量':'max'}).show()

"""输出"""
+----+------------------+---------------+---------------+
|  年|    avg(日冻结电量)  |min(日冻结电量)  |max(日冻结电量)  |
+----+------------------+---------------+---------------+
|2018|32594.561900826477|         521.12|      422371.44|
|2019| 47396.74864682531|            3.4|     1050990.15|
|2020| 71951.65986754964|        2230.56|     1376460.44|
|2016| 5774.134211900422|           34.6|      69464.032|
|2017| 18288.82305388127|          252.8|       137408.1|
+----+------------------+---------------+---------------+

+----+---------------+
|  年|max(日冻结电量)|
+----+---------------+
|2018|      422371.44|
|2019|     1050990.15|
|2020|     1376460.44|
```

```
|2016|       69464.032|
|2017|        137408.1|
+----+----------------+

```

4.5.4 Pivot()

- 行转多列如果一个有三列值的表格使用了分组后再进行pivot转换，那么将会有一列的值会变为转换后的列名。

```
#Pivot() #行转多列,
#如果一个有三列值的表格使用了分组后再进行pivot转换
#，那么将会有一列的值会变为转换后的列名。
df_pivot = df.select('年','月','日冻结电量').limit(5) # 只选了5个
df_pivot.show(5)
df_pivot = df_pivot.groupby('年').pivot('月').sum('日冻结电量')
df_pivot.show()

"""输出"""
+----+---+----------+
|  年| 月|日冻结电量|
+----+---+----------+
|2018|  3|   1291.71|
|2016| 10|   4154.02|
|2016|  6|   4040.13|
|2018|  6|   51109.7|
|2016|  1|    1353.5|
+----+---+----------+

+----+------+-------+-------+-------+
|  年|     1|      3|      6|     10|
+----+------+-------+-------+-------+
|2018|  null|1291.71|51109.7|   null|
|2016|1353.5|   null|4040.13|4154.02|
+----+------+-------+-------+-------+
```

4.6 删除的操作

4.6.1 drop()

- 删除指定的列

```python
#删除列
df_drop = df[df.columns[:3]]
print(df_drop.columns)
print(df_drop.drop('年').columns)
# 或者
print(df_drop.drop(df_drop.年).columns)


"""输出"""
['日期', '年', '月']
['日期', '月']
['日期', '月']
```

4.6.2 sparkDataFrame.na.drop()

- 按行将行中含有na的整行删除

```python
# 按行将行中含有na的整行删除
df_drop = df[['年','区域','等级']]
df_drop.show(3)
print(df_drop.count())


df_drop.na.drop().show(3)
print(df_drop.na.drop().count())

"""输出"""
+----+------+----+
|  年|  区域|等级|
+----+------+----+
|2018|皋兰县|null|
|2016|西固区|null|
|2016|榆中县|null|
+----+------+----+
only showing top 3 rows

14481
+----+--------+----+
|  年|    区域|等级|
+----+--------+----+
|2018|  永登县|  黄|
|2017|兰州新区|  黄|
|2018|  永登县|  黄|
+----+--------+----+
only showing top 3 rows

2322
```

4.6.3 sparkDataFrame.dropna()

- 指定删除任一一列或者多列包含na的行

```
# 指定删除任一一列或者多列包含na的行
df_drop = df[['年','区域','等级','分类']]
df_drop.show(3)
print(df_drop.count())

df_drop.dropna(subset=['等级','分类']).show(3)
print(df_drop.dropna(subset=['等级','分类']).count())

"""输出"""
+----+------+----+----+
|  年|  区域|等级|分类|
+----+------+----+----+
|2018|皋兰县|null|null|
|2016|西固区|null|null|
|2016|榆中县|null|null|
+----+------+----+----+
only showing top 3 rows

14481
+----+--------+----+--------+
|  年|    区域|等级|    分类|
+----+--------+----+--------+
|2018|  永登县|  黄|道路结冰|
|2017|兰州新区|  黄|    雷电|
|2018|  永登县|  黄|道路结冰|
+----+--------+----+--------+
only showing top 3 rows

2322
```

4.6.4 fillna()、 .na.fill()

- 填充空值

```
# fillna() 填充空值,与df.na.fill()相同
df_fillna = df[['年','区域','等级','分类']].limit(5)
df_fillna.show()
#将所有为na的值填充为  "空值", 可指定某列的值填充为均值或其他统计指标。
df_fillna.fillna("空值").show()
df_fillna.na.fill("空值").show()

"""输出"""
+----+--------+----+----+
|  年|    区域|等级|分类|
+----+--------+----+----+
|2018|  皋兰县|null|null|
|2016|  西固区|null|null|
|2016|  榆中县|null|null|
|2018|七里河区|null|null|
|2016|  安宁区|null|null|
+----+--------+----+----+
```

```

+----+--------+----+----+
|  年|    区域|等级|分类|
+----+--------+----+----+
|2018|  皋兰县|空值|空值|
|2016|  西固区|空值|空值|
|2016|  榆中县|空值|空值|
|2018|七里河区|空值|空值|
|2016|  安宁区|空值|空值|
+----+--------+----+----+

+----+--------+----+----+
|  年|    区域|等级|分类|
+----+--------+----+----+
|2018|  皋兰县|空值|空值|
|2016|  西固区|空值|空值|
|2016|  榆中县|空值|空值|
|2018|七里河区|空值|空值|
|2016|  安宁区|空值|空值|
+----+--------+----+----+
```

- 利用统计指标填充空值（只做演示用，年的平均值填了是没有意义的）

```
# 利用统计指标填充空值
df_na_fill = df[['年','区域','等级','分类']].limit(5)
df_na_fill.show()


max_ = df_na_fill.select(psf.max(df_na_fill.年))
max_ = str(max_.toPandas().values[0][0])
print('年的最大值',max_)

means_ = df_na_fill.select(psf.mean(df_na_fill.年))
means_ = str(means_.toPandas().values[0][0])
print('年的平均值',means_)

# 用字段'年'的最大值和平均值，分别去填充 字段 “等级”“分类”
# 这里只是演示能这么做
df_na_fill.na.fill(
    {"等级":max_
     ,"分类":means_})\
.show()

"""输出"""
+----+--------+----+----+
|  年|    区域|等级|分类|
+----+--------+----+----+
|2018|  皋兰县|null|null|
|2016|  西固区|null|null|
|2016|  榆中县|null|null|
|2018|七里河区|null|null|
|2016|  安宁区|null|null|
```

```
+----+--------+----+----+

年的最大值 2018
年的平均值 2016.8
+----+--------+----+------+
|  年|     区域|等级|  分类|
+----+--------+----+------+
|2018|  皋兰县|2018|2016.8|
|2016|  西固区|2018|2016.8|
|2016|  榆中县|2018|2016.8|
|2018|七里河区|2018|2016.8|
|2016|  安宁区|2018|2016.8|
+----+--------+----+------+
```

4.7 格式转换的操作

4.7.1 Pandas和Spark的DataFrame的相互转换

4.7.1.1 pandas 转spark

- sqlContext.createDataFrame
- spark转化为pandas，但是该数据要读入内存，如果数据量大的话，很难跑得动

  两者的优缺点:

  1. Pyspark DataFrame是在分布式节点上运行一些数据操作，而pandas是不可能的；
  2. Pyspark DataFrame的数据反映比较缓慢，没有Pandas那么及时反映；
  3. Pyspark DataFrame的数据框是不可变的，不能任意添加列，只能通过合并进行；
  4. pandas比Pyspark DataFrame有更多方便的操作以及很强大

```
#pandas转spark
spark_df = sqlContext.createDataFrame(pandas_df)
print(type(spark_df))

"""输出"""
<class 'pyspark.sql.dataframe.DataFrame'>
```

4.7.1.2 spark转pandas

- toPandas()

```
pandas_df = spark_df.toPandas()
print(type(pandas_df))

"""输出"""
<class 'pandas.core.frame.DataFrame'>
```

4.7.2 RDD与Spark dataframe转换

sparkdataframe转RDD

- sparkDataFrame.rdd

```
rdd_df = spark_df.rdd
print(type(rdd_df))

"""输出"""
<class 'pyspark.rdd.RDD'>
```

4.7.2.1 RDD转sparkdataframe

- rdd.toDF()

```
saprk_df = rdd_df.toDF()
print(type(saprk_df))

"""输出"""
<class 'pyspark.sql.dataframe.DataFrame'>
```

4.8 sql操作

- 如果要从hive中读取数据，就要在开始的时候创建一个sparksession的对象。

```
from pyspark.sql import SparkSession

spark = SparkSession\
    .builder\
    .appName('python spark sql basic example')\
    .config("spark.some.sonfig.option",'some-value') \
    .enableHiveSupport()\
    .getOrCreate()
```

  指的是如果当前存在一个SparkSession就直接获取，否组新建，enableHiveSupport()使我们可以从读取或写入数据到HIVE

- 或者我们还可以自定义配置，例如定义spark任务执行时的内存：

```
# 或者我们还可以自定义配置，例如定义spark任务执行时的内存：
from pyspark.sql import SparkSession
myspark = SparkSession.builder \
    .appName('compute_customer_age') \
    .config('spark.executor.memory','2g') \
    .enableHiveSupport() \
    .getOrCreate()
```

- 创建完sparksession对象之后，就可以从hive中取数据，进行操作。

```
sql = """
      SELECT id as customer_id,name, register_date
      FROM [db_name].[hive_table_name]
      limit 100
      """
df = myspark.sql(sql)
df.show(20)
```

- 将数据保存到数据库中可以使用下面的语句来实现：

```
DataFrame.write.mode("overwrite").saveAsTable("test_db.test_table2")
```

4.9 读写csv的操作

- 可以使用SQLContext类中 load/save函数来读取和保存CSV文件：

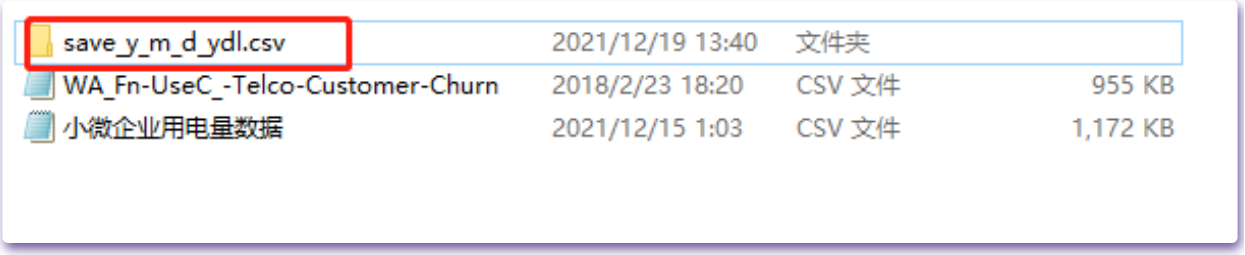
```
sc.stop()
# 可以使用SQLContext类中 load/save函数来读取和保存CSV文件：
from pyspark import SparkContext
from pyspark.sql import SQLContext

sc = SparkContext()
sqlContext = SQLContext(sc)
csv_content = sqlContext.read\
        .format('com.databricks.spark.csv')\
        .options(header='true', inferschema='true')\
        .load(r'./数据/小微企业用电量数据.csv') # 加载数据
csv_content.show(2)  #读取

"""输出"""
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------+------------+---+-----+---+
|      日期|  年| 月|季度|周(年度)|星期| 日|  城市|  区域|分类|等级|日冻结电量|小微企业数量|第一产业数量|第二产业数量|第三产业数量|year|month|day|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------+------------+---+-----+---+
|2018-03-12|2018|  3|   1|      11|   0| 12|兰州市|皋兰县|null|null|  1291.71|          16|           2|           7|           7|2018|    3| 12|
|2016-10-26|2016| 10|   4|      43|   2| 26|兰州市|西固区|null|null|  4154.02|          15|           0|           3|          12|2016|   10| 26|
+----------+----+---+----+--------+----+---+------+------+----+----+----------+------------+------------+------------+------------+---+-----+---+
only showing top 2 rows
```

- save存储数据

```
csv_content.select('year','month','day','日冻结电量')\
        .write.format('com.databricks.spark.csv')\
        .option('header','true').save('./数据/save_y_m_d_ydl.csv')
```



- 使用sparksession中的方法进行读取csv文件：

```
#使用sparksession中的方法进行读取csv文件：
from pyspark.sql.session import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.csv('./数据/save_y_m_d_ydl.csv',header=True)
df.show(5)

"""输出"""
+----+-----+---+----------+
|year|month|day|日冻结电量|
+----+-----+---+----------+
|2018|    3| 12|   1291.71|
|2016|   10| 26|   4154.02|
|2016|    6| 18|   4040.13|
|2018|    6| 30|   51109.7|
|2016|    1| 10|    1353.5|
+----+-----+---+----------+
only showing top 5 rows
```

4.10 pyspark.sql.functions

4.10.1 udf针对一列操作

- 进行自定义函数的使用，可以一次执行一行遍历实现自定义函数的功能

```
#进行自定义函数的使用，可以一次执行一行遍历实现自定义函数的功能
import pyspark.sql.functions as psf
from pyspark.sql.types import StringType #函数返回值的类型，要注意原来的数据类型是什么，注意保持一致

df_udf = df.select('year')
def trans_(x):
    if x >= 2017:
        return 'yes'
```

```
    return "no"
func = psf.udf(trans_,returnType=StringType()) # 注册函数
df_udf = df_udf.withColumn('trans_',func(df_udf.year)) # 调用函数
df_udf.select('year','trans_').show(10)

"""输出"""
+----+------+
|year|trans_|
+----+------+
|2018|   yes|
|2016|    no|
|2016|    no|
|2018|   yes|
|2016|    no|
|2017|   yes|
|2018|   yes|
|2019|   yes|
|2017|   yes|
|2017|   yes|
+----+------+
only showing top 10 rows
```

4.10.2 udf()针对多列操作

- 针对两列操作（备注：以下操作没有实际意义，只作演示）

```
from pyspark.sql import functions as psf
def generate_udf(constant_var):
    def test(col1, col2):
        if col1 == 2018 and col2 == '皋兰县':
            return "2018皋兰"
        else:
            return constant_var
    return psf.udf(test, StringType())

df.select('年','区域').withColumn('new_column'
              ,generate_udf('非2018皋兰')(psf.col('年'), psf.col('区域')))\
.show(3)

"""输出"""
+----+------+----------+
|  年|  区域|new_column|
+----+------+----------+
|2018|皋兰县|  2018皋兰|
|2016|西固区|非2018皋兰|
|2016|榆中县|非2018皋兰|
+----+------+----------+
only showing top 3 rows
```

- 使用udf对性能会有负面的影响，如果不是太过于复杂的逻辑，可以使用

f.when.when.otherwise()的方式得出想要的结果。

```
df.select(df.年
          ,df.区域
          ,psf.when((df.年 == 2018) & (df.区域 == '皋兰县'),"2018皋兰")\
          .otherwise('非2018皋兰').alias('new_column')
         ).show(3)

"""输出"""
+----+------+----------+
|  年|  区域|new_column|
+----+------+----------+
|2018|皋兰县|  2018皋兰|
|2016|西固区|非2018皋兰|
|2016|榆中县|非2018皋兰|
+----+------+----------+
only showing top 3 rows
```

4.11 字符串方法

4.11.1 concat、concat_ws

- 数据

```
from pyspark.sql.functions import concat, concat_ws
df1 = df.select('城市','区域')
df1.show(3)

"""输出"""
+------+------+
|  城市|  区域|
+------+------+
|兰州市|皋兰县|
|兰州市|西固区|
|兰州市|榆中县|
+------+------+
only showing top 3 rows
```

4.11.1.1 concat

- 直接拼接

```
# 直接拼接
df1.select('城市','区域',concat(df.城市,df.区域).alias('位置')).show(3)

"""输出"""
+------+------+------------+
|  城市|  区域|        位置|
+------+------+------------+
|兰州市|皋兰县|兰州市皋兰县|
|兰州市|西固区|兰州市西固区|
|兰州市|榆中县|兰州市榆中县|
+------+------+------------+
only showing top 3 rows
```

4.11.1.2 concat_ws

- 指定拼接符号(多个字段)

```
#指定拼接符
df1.select('城市','区域',concat_ws('-',df.城市,df.区域).alias('位置')).show(3)

"""输出"""
+------+------+-------------+
|  城市|  区域|         位置|
+------+------+-------------+
|兰州市|皋兰县|兰州市-皋兰县|
|兰州市|西固区|兰州市-西固区|
|兰州市|榆中县|兰州市-榆中县|
+------+------+-------------+
only showing top 3 rows
```

- 指定拼接符号(单个字段，则还是原来的字段)

```
#指定拼接符
df1.select('城市','区域',concat_ws('-',df.城市).alias('位置')).show(3)

"""输出"""
+------+------+-------------+
|  城市|  区域|         位置|
+------+------+-------------+
|兰州市| 皋兰县|       兰州市 |
|兰州市| 西固区|       兰州市 |
|兰州市| 榆中县|       兰州市 |
+------+------+-------------+
only showing top 3 rows
```

4.11.2 格式化字符串

- 格式化字符串

```
from pyspark.sql.functions import format_string

df.select('年'
          ,'城市'
          ,'区域'
          ,format_string('%d - %s - %s' # “-” 可以不加
                         ,df.年
                         ,df.城市
                         ,df.区域)\
              .alias('格式化字符串')
         )\
.show()

"""输出"""
+----+------+--------+-----------------------+
|  年|  城市|    区域|             格式化字符串|
+----+------+--------+-----------------------+
|2018|兰州市|  皋兰县|  2018 - 兰州市 - 皋兰县|
|2016|兰州市|  西固区|  2016 - 兰州市 - 西固区|
|2016|兰州市|  榆中县|  2016 - 兰州市 - 榆中县|
|2018|兰州市|七里河区|2018 - 兰州市 - 七里河区|
|2016|兰州市|  安宁区|  2016 - 兰州市 - 安宁区|
|2017|兰州市|  红古区|  2017 - 兰州市 - 红古区|
|2018|兰州市|  永登县|  2018 - 兰州市 - 永登县|
|2019|兰州市|  永登县|  2019 - 兰州市 - 永登县|
|2017|兰州市|  西固区|  2017 - 兰州市 - 西固区|
|2017|兰州市|兰州新区|2017 - 兰州市 - 兰州新区|
|2018|兰州市|  西固区|  2018 - 兰州市 - 西固区|
|2020|兰州市|七里河区|2020 - 兰州市 - 七里河区|
|2018|兰州市|兰州新区|2018 - 兰州市 - 兰州新区|
|2019|兰州市|  城关区|  2019 - 兰州市 - 城关区|
|2018|兰州市|七里河区|2018 - 兰州市 - 七里河区|
|2019|兰州市|  永登县|  2019 - 兰州市 - 永登县|
|2020|兰州市|  安宁区|  2020 - 兰州市 - 安宁区|
|2017|兰州市|  红古区|  2017 - 兰州市 - 红古区|
|2016|兰州市|七里河区|2016 - 兰州市 - 七里河区|
|2016|兰州市|  安宁区|  2016 - 兰州市 - 安宁区|
+----+------+--------+-----------------------+
only showing top 20 rows
```

4.11.3 instr

- 查找字符串的位置

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import instr

spark = SparkSession.builder.getOrCreate()
data = spark.createDataFrame([('abcd',)], ['s'])
data.show()
data.select(instr(data.s, 'b').alias('s')).show()
# 注意这里返回的索引是从1开始数，如果找不到才返回0
```

```

"""输出"""
+----+
|   s|
+----+
|abcd|
+----+

+---+
|  s|
+---+
|  2|
+---+
```

4.11.4 substring

- 字符串截取

```
from pyspark.sql.functions import substring

data.show()
data.select(substring(data.s, 1, 2).alias('s')).show()  #1与2表示开始与截取长度

"""输出"""
+----+
|   s|
+----+
|abcd|
+----+

+---+
|  s|
+---+
| ab|
+---+

```

4.11.5 regexp_repalce

- 正则表达式替换

```
# 正则表达式替换(将数字替换为“-”)
from pyspark.sql.functions import regexp_replace
data = spark.createDataFrame([('100sss200',)], ['str'])
data.show()
data.select(regexp_replace('str', '(\d)', '-').alias('d')).show()
#替换类型，正则语句，替换内容

"""输出"""
+---------+
|      str|
```

```
+---------+
|100sss200|
+---------+

+---------+
|        d|
+---------+
|---sss---|
+---------+
```

4.12 与时间相关的方法

4.12.1 格式更改、日期获取

- 时间格式更改：使用pyspark.sql.functions.date_format方法

```
#将时间格式进行更改:
#使用pyspark.sql.functions.date_format方法
from pyspark.sql import functions as F
df = spark.createDataFrame([('2021-12-08',)], ['dt'])
print(df.collect())
df.select(F.date_format('dt', 'yyyyMMdd').alias('date')).collect()

"""输出"""
[Row(dt='2021-12-08')]
[Row(date='20211208')]
```

- 获取当前日期

```
from pyspark.sql.functions import current_date

spark.range(3).withColumn('date',current_date()).show()

"""输出"""
+---+----------+
| id|      date|
+---+----------+
|  0|2021-12-20|
|  1|2021-12-20|
|  2|2021-12-20|
+---+----------+
```

- 获取当前日期时间

```
from pyspark.sql.functions import current_timestamp
spark.range(3).withColumn('date',current_timestamp()).show()
spark.range(3).withColumn('date',current_timestamp()).collect()

"""输出"""
+---+--------------------+
```

```
| id|                 date|
+---+--------------------+
|  0|2021-12-20 00:18:...|
|  1|2021-12-20 00:18:...|
|  2|2021-12-20 00:18:...|
+---+--------------------+

[Row(id=0, date=datetime.datetime(2021, 12, 20, 0, 18, 49, 370000)),
 Row(id=1, date=datetime.datetime(2021, 12, 20, 0, 18, 49, 370000)),
 Row(id=2, date=datetime.datetime(2021, 12, 20, 0, 18, 49, 370000))]
```

- 将字符串日期改为时间日期格式：

```
from pyspark.sql.functions import to_date, to_timestamp
df = spark.createDataFrame([('1997-02-28 10:30:00',)], ['t'])
df.show()
df.select(to_date(df.t).alias('date')).show()   # 1.转日期
df.select(to_timestamp(df.t).alias('dt')).show()   # 2.带时间的日期
df.select(to_timestamp(df.t, 'yyyy-MM-dd HH:mm:ss').alias('dt')).show()   # 3.可以指定日期格式


"""输出"""
+-------------------+
|                  t|
+-------------------+
|1997-02-28 10:30:00|
+-------------------+

+----------+
|      date|
+----------+
|1997-02-28|
+----------+

+-------------------+
|                 dt|
+-------------------+
|1997-02-28 10:30:00|
+-------------------+

+-------------------+
|                 dt|
+-------------------+
|1997-02-28 10:30:00|
+-------------------+
```

- 获取日期中的年月日

```python
from pyspark.sql.functions import year, month, dayofmonth
df = spark.createDataFrame([('2015-04-08',)], ['a'])
df.show()
df.select(year('a').alias('year'),
          month('a').alias('month'),
          dayofmonth('a').alias('day')
  ).show()
#只接受以-连接的日期，如果是以/等其他连接符连接的，需要进行格式转换或者字符替换。

"""输出"""
+----------+
|         a|
+----------+
|2015-04-08|
+----------+

+----+-----+---+
|year|month|day|
+----+-----+---+
|2015|    4|  8|
+----+-----+---+
```

4.12.2 日期、月份差

- 日期差

```python
from pyspark.sql.functions import datediff, months_between


df = spark.createDataFrame([('2015-04-08','2015-05-10')], ['d1', 'd2'])
df.show()
df.select(datediff(df.d2, df.d1).alias('diff')).show()


"""输出"""
+----------+----------+
|        d1|        d2|
+----------+----------+
|2015-04-08|2015-05-10|
+----------+----------+

+----+
|diff|
+----+
|  32|
+----+
```

- 月份差

```
df = spark.createDataFrame([('1997-02-28 10:30:00', '1996-10-30')], ['t', 'd'])
df.show()
df.select(months_between(df.t, df.d).alias('months')).show()

"""输出"""
+-------------------+----------+
|                  t|         d|
+-------------------+----------+
|1997-02-28 10:30:00|1996-10-30|
+-------------------+----------+

+----------+
|    months|
+----------+
|3.94959677|
+----------+
```

4.13 窗口函数的使用

- Window

```
sc.stop()

from pyspark.sql import Row, functions as F
from pyspark.sql.window import Window
from pyspark.sql import SQLContext

sc = SparkContext("local", "count app")
rddData = sc.parallelize( (Row(c="class1", s=50), Row(c="class2", s=40),\
                           Row(c="class3", s=70), Row(c="class2", s=49),\
                           Row(c="class3", s=29), Row(c="class1", s=78)))
sqlContext = SQLContext(sc)
testDF = rddData.toDF()
testDF.show()
result = (testDF.select("c", "s", F.row_number().over(Window.partitionBy("c").orderBy("s")).alias("rowNum")))
result.show()
finalResult = result.where(result.rowNum <= 1).show()

"""输出"""
+------+---+
|     c|  s|
+------+---+
|class1| 50|
|class2| 40|
```

```
|class3| 70|
|class2| 49|
|class3| 29|
|class1| 78|
+------+---+

+------+---+------+
|     c|  s|rowNum|
+------+---+------+
|class2| 40|     1|
|class2| 49|     2|
|class1| 50|     1|
|class1| 78|     2|
|class3| 29|     1|
|class3| 70|     2|
+------+---+------+

+------+---+------+
|     c|  s|rowNum|
+------+---+------+
|class2| 40|     1|
|class1| 50|     1|
|class3| 29|     1|
+------+---+------+
```

4.14 其他函数

- psf中其他常用的函数

```
pyspark.sql.functions.abs(col)    #计算绝对值
pyspark.sql.functions.avg(col)    #聚合函数：返回组中的值的平均值。
pyspark.sql.functions.variance(col)    #返回组中值的总体方差
pyspark.sql.functions.ceil(col)    #计算给定值的上限
pyspark.sql.functions.floor(col)    #计算给定值的下限。
pyspark.sql.functions.collect_list(col)    #返回重复对象的列表。
pyspark.sql.functions.collect_set(col)    #返回一组消除重复元素的对象。
pyspark.sql.functions.count(col)    #返回组中的项数量。
pyspark.sql.functions.countDistinct(col, *cols)    #返回一列或多列的去重计数的新列。
pyspark.sql.functions.initcap(col)    #在句子中将每个单词的第一个字母翻译成大写。
pyspark.sql.functions.isnan(col)    #如果列是NaN，则返回true的表达式
pyspark.sql.functions.lit(col)    #创建一个文字值的列
pyspark.sql.functions.lower(col)    #将字符串列转换为小写
pyspark.sql.functions.reverse(col)    #反转字符串列并将其作为新的字符串列返回
pyspark.sql.functions.sort_array(col, asc=True)    #按升序对给定列的输入数组进行排序
pyspark.sql.functions.split(str, pattern)    #按指定字符进行分隔数据
pyspark.sql.functions. array_min (col)    #计算指定列的最小值
```

```
pyspark.sql.functions. array_max (col)    #计算指定列的最大值
pyspark.sql.functions.stddev(col)     #   返回组中表达式的无偏样本标准差
pyspark.sql.functions.sumDistinct(col)    #返回表达式中不同值的总和
pyspark.sql.functions.trim(col)    #去除空格
pyspark.sql.functions. greatest (col1,col2)    #求行的最大值，可以计算一行中多列的最大值
pyspark.sql.functions. least (col1,col2)    #求行的最小值，可以计算一行中多列的最小值，也可以用lit()指定常数进行与列的值进行比较

```